

Department of Electronics and Communications Engineering
National Institute of Technology Rourkela, Odisha, India



PROJECT REPORT ON RISC-V SINGLE CYCLE CORE

PROF.AYAS KANTA SWAIN

SUBMITTED BY:-
ASHUTOSH PADARABINDA(121EI0375)

CONTENTS

• INTRODUCTION.....	3
• ALU DESIGN.....	5
• CONTROL UNIT.....	6
• DATA PATH.....	11
• VERILOG MODULES.....	17
• REFERENCES.....	24

INTRODUCTION

ABOUT RISC-V

The world of processor design has long been dominated by a few major players with proprietary instruction set architectures (ISAs). This closed ecosystem limited innovation and customization for specific applications. However, a new era has dawned with the rise of RISC-V, an open-source ISA poised to revolutionize processor design. This essay will explore the core principles of RISC-V design, its advantages, and the potential impact it holds for the future of computing.

At its heart, RISC-V follows the philosophy of Reduced Instruction Set Computing (RISC). Unlike its counterpart, Complex Instruction Set Computing (CISC), RISC-V utilizes a smaller set of simpler instructions. This streamlined approach allows for efficient decoding and execution, leading to faster processors with lower power consumption. Furthermore, the open-source nature of RISC-V fosters collaboration and innovation. Anyone can access, modify, and develop custom extensions to the base ISA, enabling the creation of processors tailored to specific needs. This flexibility is particularly valuable for niche applications like embedded systems and Internet-of-Things (IoT) devices, where power efficiency and low cost are paramount.

HOW TO DESIGN RISC-V PROCESSOR

- Choose the RISC-V ISA configuration.
- Design the processor microarchitecture.
- Instruction Set Decoding logic.
- Design of ALU.
- Design Control Unit.
- Design of datapath.
- Design of memory hierarchy.
- Verification and Validation.

ADVANTAGES OF RISC-V

RISC-V processors offer several advantages over traditional architectures:

- **Open-source and royalty-free:** Unlike proprietary ISAs, RISC-V is open-source, allowing anyone to design, modify, and distribute RISC-V cores without paying licensing fees. This fosters innovation and lowers the barrier to entry for new players.
- **Flexibility and Customization:** The modular design of RISC-V with its base ISA and optional extensions enables customization for specific needs. You can choose the exact features required for your application, leading to optimized performance, power efficiency, and cost-effectiveness.

- **Simplicity and Efficiency:** RISC-V follows the RISC principles of a small set of simple instructions. This streamlined approach simplifies processor design, reduces complexity, and potentially leads to faster execution and lower power consumption.
- **Security:** The open nature of RISC-V allows for greater scrutiny and community-driven security enhancements. Additionally, specific extensions like RV32C can be incorporated for hardware-based security features.
- **Scalability:** RISC-V can be scaled to address a wide range of applications. The same ISA can be used for tiny embedded devices requiring minimal power to high-performance processors needing complex functionalities.
- **Vibrant Ecosystem:** The open-source nature fosters collaboration and a growing ecosystem of developers, researchers, and toolmakers. This collaborative environment accelerates innovation and development for both hardware and software aspects of RISC-V.
- **Future-proof:** With increasing focus on open-source hardware and diverse computing needs, RISC-V is well-positioned for future advancements. Its flexibility allows for adaptation to evolving technologies and application demands.

THE ALU

We will design an ALU that can perform a subset of the ALU operations of a full Processor ALU. In this exercise, we will develop an ALU that will take two 2-inputs, A and B and is able to execute the following instructions:

000 (add)	add
001 (subtract)	sub
101 (set less than)	slt
011 (or)	or
010 (and)	and

FIG: ALU OPCODE

The ALU will generate a 32-bit output that we will call 'Result' and an additional 1-bit flag 'Zero' that will be set to 'logic-1' if all the bits of 'Result' are 0. The different operations will be selected by a 3-bit control signal called 'ALUControl' according to the following table. For example, when the 'ALUControl' input is '011', the function $\text{Result} = A \text{ or } B$ should be calculated. It is easy to see that there are many values of 'ALUControl' for which no operation has been defined. It is not very important what the circuit does when 'ALUControl' has these values, since the 'Result' will simply be ignored in these cases. You can use this to your advantage to simplify the circuit. Right now, the described operations may look random, but once we learn more about the Instruction set architecture, these choices will make more sense.

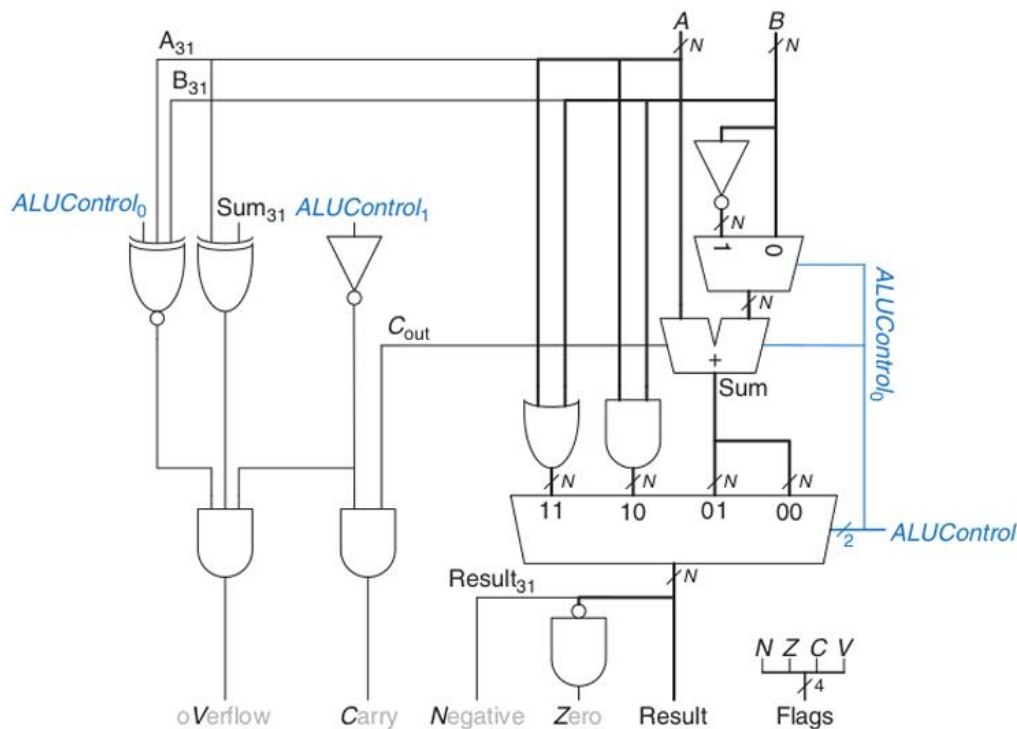


FIG: ALU LOGIC DIAGRAM

The three instructions add, sub, and slt are arithmetic operations, whereas the two remaining and, or are logical operations. Therefore, we have two separate groups of operations. Now let us look at the figure above and determine for which values of ALUControl we perform an operation from which group. It should be pretty clear that when ALUControl[1] is logic-1 we select a logic operation and when ALUControl[1] is logic-0 we have an arithmetic operation. This means that the output of either group can be selected by a 4-input multiplexer that is controlled by ALUControl[1:0].

Note that in the above example, there are many values of ALUControl where the circuit would perform ‘strange’ operations (100 for example). This is not important because the circuit specification that was given to us said that these inputs were not relevant (this is probably because it can be guaranteed that these inputs will not appear during normal operation)

THE CONTROL UNIT

RISC-V uses 32-bit instructions. RISC-V consists of defining the following instruction formats: R-type, I-type, S-Type, B-Type. R-type instructions operate on three registers. I-type, S-type and B-type instructions operate on two registers and a 12-bit immediate.

THE R-TYPE INSTRUCTION

The name R-type is short for register-type. R-type instructions use three registers as operands: two as sources, and one as a destination. The figure below shows the R-type machine instruction format. The 32-bit instruction has six fields: op, rs1, rs2, rd, funct3, and funct7. Each field is five or seven bits, as indicated. The operation the instruction performs is encoded in the three fields highlighted in blue: op (also called opcode or operation code) and funct3 and funct7 (also called the function). All R-type instructions have an opcode of 33. The specific R-type operation is determined by the function fields. The operands are encoded in the three fields: rs1, rs2, and rd. The first two registers, rs1, and rs2 are the source registers; rd is the destination register.

Instruction Format for some of the R-type instructions is shown below:

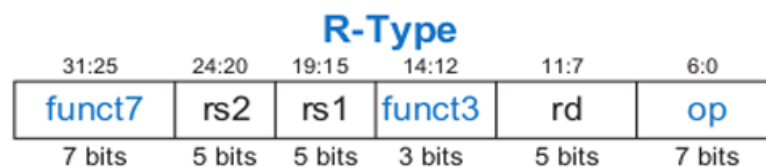


FIG: R-TYPE INSTRUCTION FORMAT

0000000	rs2	rs1	000	rd	0110011	ADD
0100000	rs2	rs1	000	rd	0110011	SUB
0000000	rs2	rs1	001	rd	0110011	SLL
0000000	rs2	rs1	010	rd	0110011	SLT
0000000	rs2	rs1	011	rd	0110011	SLTU
0000000	rs2	rs1	100	rd	0110011	XOR
0000000	rs2	rs1	101	rd	0110011	SRL
0100000	rs2	rs1	101	rd	0110011	SRA
0000000	rs2	rs1	110	rd	0110011	OR
0000000	rs2	rs1	111	rd	0110011	AND

FIG: SOME R-TYPE INSTRUCTIONS

THE I-TYPE INSTRUCTION

The name I-type is short for immediate-type. I-type instructions use two register operands and one immediate operand. The figure below shows the I-type machine instruction format. The 32-bit instruction has five fields: op, rs1, rd, funct3, and imm. The first three fields, op, rs1, and rd, are like those of R-type instructions. The imm field holds the 12-bit immediate. The funct3 holds the operation to be performed. The operation is determined by the opcode and funct3, highlighted in blue. The operands are specified in the two fields rs1, and imm. rs1 and imm are always used as source operands. rd is used as a destination.



FIG: I-TYPE INSTRUCTION FORMAT

Instruction format for some of the I-type instructions are shown below:

imm _{11:0}		rs1	000	rd	0000011	LB
imm _{11:0}		rs1	001	rd	0000011	LH
imm _{11:0}		rs1	010	rd	0000011	LW
imm _{11:0}		rs1	100	rd	0000011	LBU
imm _{11:0}		rs1	101	rd	0000011	LHU
imm _{11:0}		rs1	000	rd	0010011	ADDI
imm _{11:0}		rs1	010	rd	0010011	SLTI
imm _{11:0}		rs1	011	rd	0010011	SLTIU
imm _{11:0}		rs1	100	rd	0010011	XORI
imm _{11:0}		rs1	110	rd	0010011	ORI
imm _{11:0}		rs1	111	rd	0010011	ANDI
0000000	shamt	rs1	001	rd	0010011	SLLI
0000000	shamt	rs1	101	rd	0010011	SRLI
0100000	shamt	rs1	101	rd	0010011	SRAI

FIG: SOME I-TYPE INSTRUCTIONS

THE S-TYPE INSTRUCTION

The name S-type is short for store-type. S-type instructions use two register operands and one immediate operand. Figure below shows the S-type machine instruction format. The 32-bit instruction has five fields: op, rs1, rs2, funct3 and imm. The first three fields op, rs1, and rs2 are like those of R-type instructions. The imm fields hold the 12-bit immediate. The 12-bit immediate is split into two sets as shown below. The operation is determined by the opcode and funct3, highlighted in blue. The operands are specified in the three fields rs1, rs2 and imm.

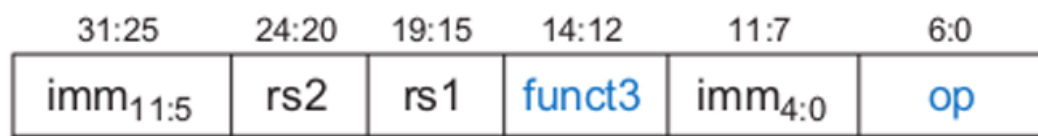


FIG: S-TYPE INSTRUCTION FORMAT

Instruction format for some of the S-type instructions are shown below:

imm[11:5]		rs2	rs1	000	imm[4:0]	0100011	SB
imm[11:5]		rs2	rs1	001	imm[4:0]	0100011	SH
imm[11:5]		rs2	rs1	010	imm[4:0]	0100011	SW

The control unit computes the control signals based on the opcode and funct fields of the instruction, Instr[31:25], Instr[14:12] and Instr[6:0]. Most of the control information comes from the opcode, but for further operations function fields are used. Thus, we will simplify our design by factoring the control unit into two blocks of combinational logic.

THE MAIN DECODER

The table below is a truth table for the main decoder that summarizes the control signals as a function of the opcode. All R-type instructions use the same main decoder values; they differ only in the ALU decoder output. Recall that, for instructions that do not write to the register file (e.g., S-type), the ResultSrc control signals is don't care (X); the address and data to the register write port do not matter because RegWrite is not asserted. The logic for the decoder can be designed using your favorite techniques for combinational logic design.

Instruction	Op	RegWrite	ImmSrc	ALUSrc	MemWrite	ResultSrc	Branch	ALUOp
lw	0000011	1	00	1	0	1	0	00
sw	0100011	0	01	1	1	x	0	00
R-type	0110011	1	xx	0	0	0	0	10

ALU DECODER

The main decoder computes most of the outputs from the opcode. It also determines a 2-bit ALUOp signal. The ALU decoder uses this ALUOp signal in conjunction with the funct field and opcode bit to compute ALUControl. The meaning of the ALUOp signal is given in Table below.

ALUOp	Meaning
00	add
01	subtract
10	look at funct fields and opcode bit

Table below is a truth table for the ALU decoder. The logic of ALUControl was covered in the above chapter. When ALUOp is 00 or 01, the ALU should add or subtract, respectively. When ALUOp is 10, the decoder examines the function fields and operand bit to determine the ALUControl. The control signals for each instruction were described as we built the datapath.

ALUOp	funct3	{op ₅ , funct7 ₅ }	ALUControl	Instruction
00	x	x	000 (add)	lw, sw
01	x	x	001 (subtract)	beq
10	000	00, 01, 10	000 (add)	add
	000	11	001 (subtract)	sub
	010	x	101 (set less than)	slt
	110	x	011 (or)	or
	111	x	010 (and)	and

DATA PATH

A good way to design a complex system is to start with hardware containing the state elements. These elements include the memories and the architectural state (the program counter and registers). Then, add blocks of combinational logic between the state elements to compute the new state based on the current state. The instruction is read from part of memory; load and store instructions then read or write data from another part of memory. Hence, it is often convenient to partition the overall memory into two smaller memories, one containing instructions and the other containing data. Figure below shows a block diagram with the four state elements: **the program counter, register file, and instruction and data memories.**

The first Datapath element we need is Instruction Memory. It is a memory unit to store the instructions of a program and supply instructions given an address. The instruction memory need only provide read access because the datapath does not write instructions. Since the instruction memory only reads, we treat it as combinational logic: the output at any time reflects the contents of the location specified by the address input, and no read control signal is needed. (We will need to write the instruction memory when we load the program; this is not hard to add, and we ignore it for simplicity.)

The second Datapath element we need is Program Counter (PC), which is a 32-bit register that holds the address of the current instruction. It is written at the end of every clock cycle and thus does not need a write control signal. We need an Adder to increment the PC to the address of the next instruction. It has been permanently made an adder and cannot perform the other ALU functions.

The third element, data memory has a single read/write port. If the write enable, WE, is 1, it writes data WD into address A on the rising edge of the clock. If the write enable is 0, it reads address A onto RD.

The fourth Datapath element we need is Register file. The processor's 32 general-purpose registers are stored in a structure called a register file. A register file is a collection of registers in which any register can be read or written by specifying the number of the register in the file. We need Alu to operate on the values read from the registers. R-format instructions have three register operands i.e. read two data words from the register file and write one data word into the register file for each instruction. For each data word to be read from the registers, we need an input to the register file that specifies the register number to be read and an output from the register file that will carry the value that has been read from the registers. To write a data word, we will need two inputs: one to specify the register number to be written and one to supply the data to be written into the register. we need a total of three inputs (two for register numbers and one for data) and two outputs (both for data). The register number inputs are 5 bits wide to specify one of 32 registers ($32 = 2^5$), whereas the data input and two data output buses are each 64 bits wide.

We also have other element, Immediate Generation. The immediate generation unit (ImmGen) has a 32-bit instruction as input that selects a 12-bit field for load, store, and branch if equal that is sign extended into a 32-bit result appearing on the output. Sign-extended means to

increase the size of a data item by replicating the high-order sign bit of the original data item in the high-order bits of the larger, destination data item.

A = 0101 0000 1101

B = 1011 1100 1111

Sign Extension of A = 0000 0000 0000 0000 0000 0101 0000 1101

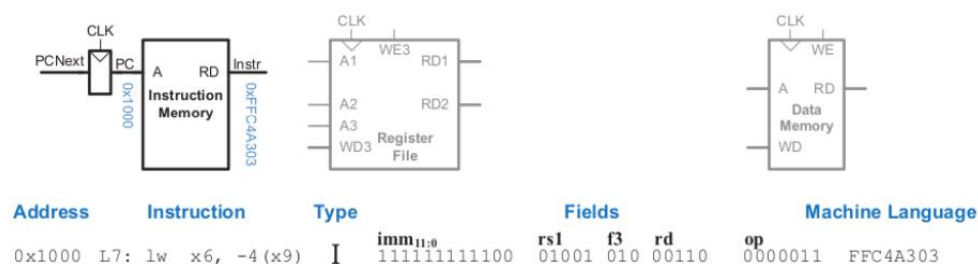
Sign Extension of B = 1111 1111 1111 1111 1111 1011 1100 1111

FIG: Sign extension example

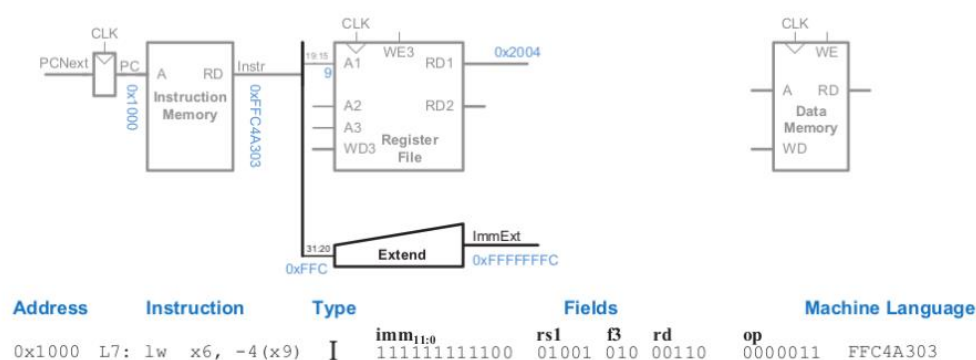
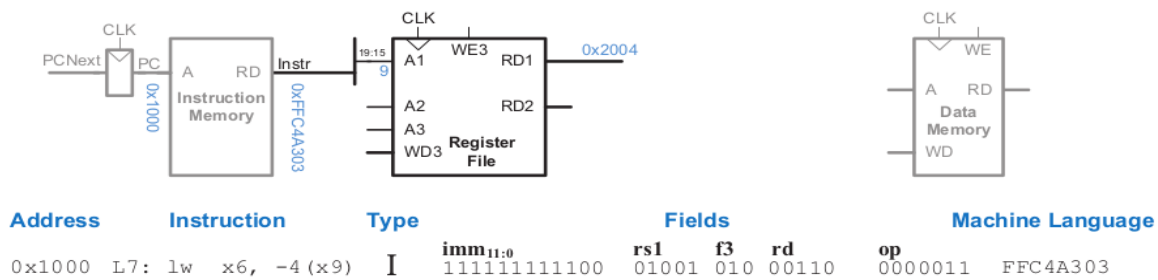
To execute any instruction, we must start by fetching the instruction from memory. To prepare for executing the next instruction, we must also increment the program counter so that it points at the next instruction, 4 bytes later. Figure below shows how to combine the three elements from Figure above to form a datapath that fetches instructions and increments the PC to obtain the address of the next sequential instruction.

DATAPATH FOR LOAD WORD INSTRUCTION

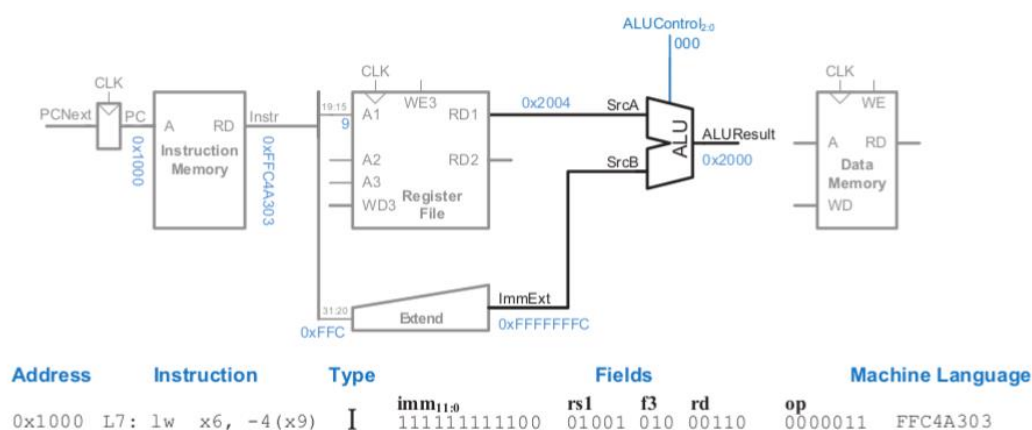
This section gradually develops the single-cycle datapath, adding one piece at a time to the state elements. The program counter (PC) register contains the address of the instruction to execute. The first step is to read the instruction from instruction memory. Figure below shows that the PC is simply connected to the address input of the instruction memory. The instruction memory reads out, or fetches, the 32-bit instruction, labeled Instr.



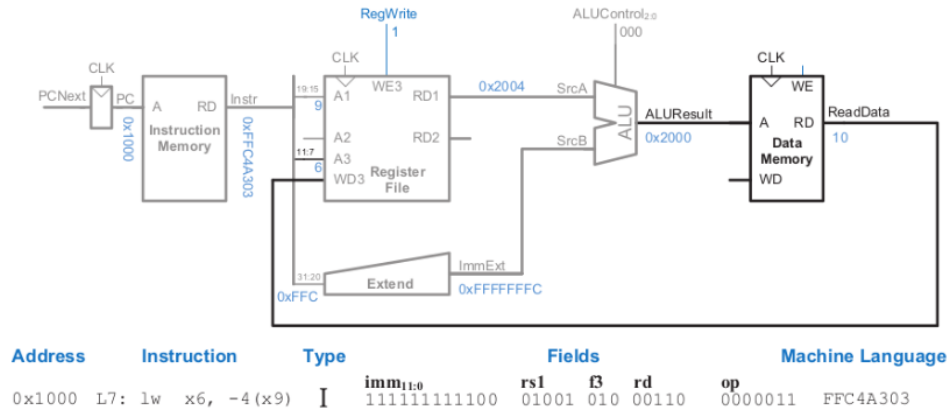
For a load instruction, the next step is to read the source register containing the base address. This register is specified in the rs1 field of the instruction, Instr[19:15]. These bits of the instruction are connected to the address input of one of the register file read ports, A1, as shown in Figure below. The register file reads the register value onto RD1.



The processor must add the base address to the offset to find the address to read from memory. Figure below introduces an ALU to perform this addition. The ALU receives two operands, SrcA and SrcB. SrcA comes from the register file, and SrcB comes from the sign-extended immediate. The ALU can perform many operations. The 3-bit ALUControl signal specifies the operation. The ALU generates a 32-bit ALUResult and a Zero flag, which indicates whether $\text{ALUResult} == 0$. For a load instruction, the ALUControl signal should be set to 000 to add the base address and offset.

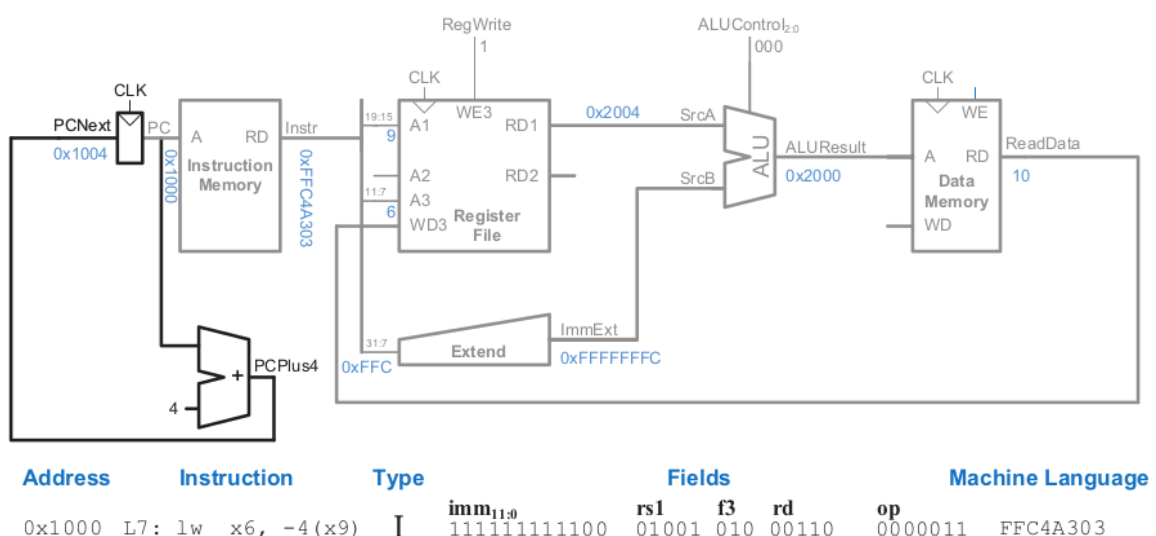


ALUResult is sent to the data memory as the address for the load instruction. The data is read from the data memory onto the ReadData bus, then written back to the destination register in the register file at the end of the cycle, as shown in Figure below. Port 3 of the register file is the write port.



The destination register for the load instruction is specified in the rd field, Instr[11:7], which is connected to the port 3 address input, A3, of the register file. The ReadData bus is connected to the port 3 write data input, WD3, of the register file. A control signal called RegWrite is connected to the port 3 write enable input, WE3, and is asserted during a load instruction so that the data value is written into the register file.

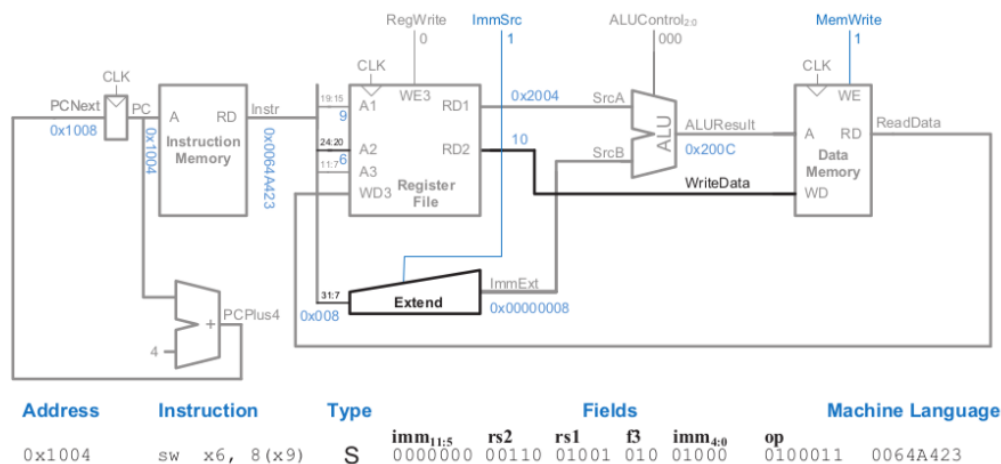
The write takes place on the rising edge of the clock at the end of the cycle. While the instruction is being executed, the processor must compute the address of the next instruction, PCNext. Because instructions are 32 bits = 4 bytes, the next instruction is at PC + 4. Figure below shows that datapath uses another adder to increment the PC by 4. The new address is written into the program counter on the next rising edge of the clock. This completes the datapath for the load instruction.



DATAPATH FOR STORE WORD INSTRUCTION

In this section, we will extend the data path to Store instructions. Like the load instruction, the store instruction reads a base address from port 1 of the register file and sign-extends an immediate. The ALU adds the base address to the immediate to find the memory address. All of these functions are already supported by the datapath. The store instruction also reads a second register from the register file and writes it to the data memory. The figure below shows the new connections for this function. The register is specified in the rs2 field, Instr[24:20]. These bits of the instruction are connected to the second register file read port, A2.

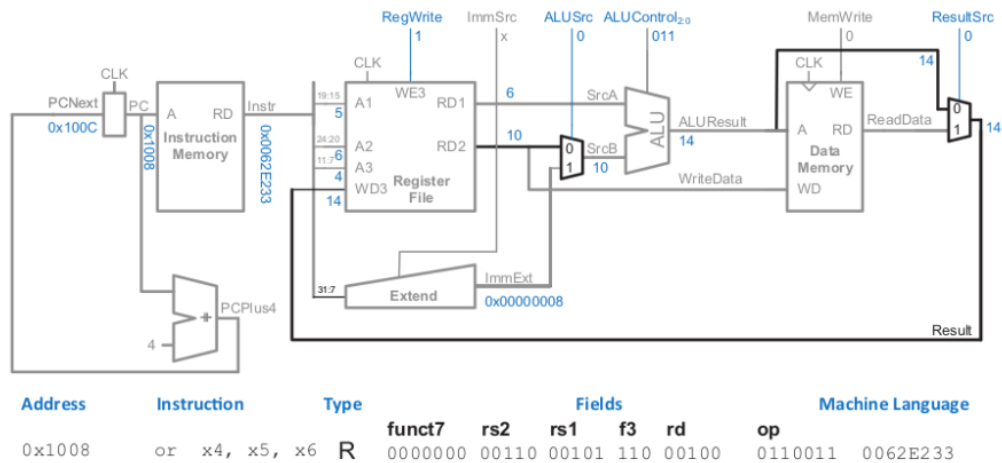
The register value is read onto the RD2 port. It is connected to the write data port of the data memory. The write enable port of the data memory, WE, are controlled by MemWrite. For a store instruction, MemWrite = 1, to write the data to memory; ALUControl = 000, to add the base address and offset; and RegWrite = 0, because nothing should be written to the register file. Note that data is still read from the address given to the data memory, but that this ReadData is ignored because RegWrite = 0.



DATAPATH FOR R TYPE INSTRUCTION

In this section we will extend the data path to handle R-type instructions add, sub, and, or, and slt. All of these instructions read two registers from the register file, perform some ALU operation on them, and write the result back to a third register in the register file. They differ only in the specific ALU operation. Hence, they can all be handled with the same hardware, using different ALUControl signals.

Figure below shows the enhanced datapath handling R-type instructions. The register file reads two registers. The ALU performs an operation on these two registers. In previous labs we saw that the ALU always received its SrcB operand from the sign-extended immediate (ImmExt). Now, we add a multiplexer to choose SrcB from either the register file RD2 port or ImmExt. The multiplexer is controlled by a new signal, ALUSrc. ALUSrc is 0 for R-type instructions to choose SrcB from the register file; it is 1 for Load and store to choose ImmExt.



This principle of enhancing the datapath's capabilities by adding a multiplexer to choose inputs from several possibilities is extremely useful. Indeed, we will apply it twice more to complete the handling of R-type instructions. Previously, the register file always got its written data from the data memory. However, R-type instructions write the ALUResult to the register file. Therefore, we add another multiplexer to choose between ReadData and ALUResult. We call its output Result. This multiplexer is controlled by another new signal, ResultSrc. ResultSrc is 0 for R-type instructions to choose Result from the ALUResult; it is 1 for Load to choose ReadData. We don't care about the value of ResultSrc for store, because store does not write to the register file.

VERILOG MODULES

ALU WITH FLAGS

```
1 module ALU(A,B,Result,ALUControl,OverFlow,Carry,Zero,Negative);
2
3     input [31:0]A,B;
4     input [2:0]ALUControl;
5     output Carry,OverFlow,Zero,Negative;
6     output [31:0]Result;
7
8     wire Cout;
9     wire [31:0]Sum;
10
11     assign {Cout,Sum} = (ALUControl[0] == 1'b0) ? A + B :
12                     (A + ((~B)+1)) ;
13     assign Result = (ALUControl == 3'b000) ? Sum :
14                     (ALUControl == 3'b001) ? Sum :
15                     (ALUControl == 3'b010) ? A & B :
16                     (ALUControl == 3'b011) ? A | B :
17                     (ALUControl == 3'b101) ? {{31{1'b0}},(Sum[31])} :
18                     {32{1'b0}};
19
20     assign OverFlow = ((Sum[31] ^ A[31]) &
21                     (~((ALUControl[0] ^ B[31] ^ A[31])) &
22                     (~ALUControl[1])));
23     assign Carry = ((~ALUControl[1]) & Cout);
24     assign Zero = &(~Result);
25     assign Negative = Result[31];
26 endmodule
```

ALU DECODER

```
1 module ALU_Decoder(ALUOp,funcnt3,funcnt7,op,ALUControl);
2
3     input [1:0]ALUOp;
4     input [2:0]funcnt3;
5     input [6:0]funcnt7,op;
6     output [2:0]ALUControl;
7     assign ALUControl = (ALUOp == 2'b00) ? 3'b000 :
8                     (ALUOp == 2'b01) ? 3'b001 :
9                     ((ALUOp == 2'b10) & (funcnt3 == 3'b000) &
10                     ({op[5],funcnt7[5]} == 2'b11)) ? 3'b001 :
11                     ((ALUOp == 2'b10) & (funcnt3 == 3'b000) &
12                     ({op[5],funcnt7[5]} != 2'b11)) ? 3'b000 :
13                     ((ALUOp == 2'b10) & (funcnt3 == 3'b010)) ?
14                     3'b101 :
15                     ((ALUOp == 2'b10) & (funcnt3 == 3'b110)) ?
16                     3'b011 :
17                     ((ALUOp == 2'b10) & (funcnt3 == 3'b111)) ?
18                     3'b010 :
19                     3'b000 ;
20 endmodule
```

MAIN DECODER

```
1 module Main_Decoder(Op,RegWrite,ImmSrc,ALUSrc,MemWrite,ResultSrc,PCSrc,ALUOp,zero);
2 //inputs and outputs decalration
3 input [6:0]Op;
4 input zero;
5 output RegWrite,ALUSrc,MemWrite,ResultSrc,PCSrc;
6 output [1:0]ImmSrc,ALUOp;
7 //interim wire declaration
8 wire Branch;
9 assign RegWrite = (Op == 7'b0000011 | Op == 7'b0110011) ? 1'b1 :
10                                                    1'b0 ;
11 assign ImmSrc = (Op == 7'b0100011) ? 2'b01 :
12                (Op == 7'b1100011) ? 2'b10 :
13                2'b00 ;
14 assign ALUSrc = (Op == 7'b0000011 | Op == 7'b0100011) ? 1'b1 :
15                                                         1'b0 ;
16 assign MemWrite = (Op == 7'b0100011) ? 1'b1 :
17                                                         1'b0 ;
18 assign ResultSrc = (Op == 7'b0000011) ? 1'b1 :
19                                                         1'b0 ;
20 assign Branch = (Op == 7'b1100011) ? 1'b1 :
21                                                         1'b0 ;
22 assign ALUOp = (Op == 7'b0110011) ? 2'b10 :
23                (Op == 7'b1100011) ? 2'b01 :
24                2'b00 ;
25
26 assign PCSrc=zero & Branch;
27
28
29 endmodule
```

CONTROL UNIT TOP

```
1 `include "ALU_Decoder.v"
2 `include "Main_Decoder.v"
3
4 module Control_Unit_Top(Op,RegWrite,ImmSrc,ALUSrc,MemWrite,ResultSrc,PCSrc,funct3,funct7,ALUControl,zero);
5
6     input [6:0]Op,funct7;
7     input [2:0]funct3;
8     input zero;
9     output RegWrite,ALUSrc,MemWrite,ResultSrc,PCSrc;
10    output [1:0]ImmSrc;
11    output [2:0]ALUControl;
12 //wire declartion
13 wire [1:0]ALUOp;
14
15    Main_Decoder Main_Decoder(
16        .Op(Op),
17        .RegWrite(RegWrite),
18        .ImmSrc(ImmSrc),
19        .MemWrite(MemWrite),
20        .ResultSrc(ResultSrc),
21        .PCSrc(PCSrc),
22        .ALUSrc(ALUSrc),
23        .ALUOp(ALUOp),
24        .zero(zero)
25    );
26
27    ALU_Decoder ALU_Decoder(
28        .ALUOp(ALUOp),
29        .funct3(funct3),
30        .funct7(funct7),
31        .op(Op),
32        .ALUControl(ALUControl)
33    );
34
35
36 endmodule
```

MUX

```
1 module Mux (a,b,s,c);
2
3     input [31:0]a,b;
4     input s;
5     output [31:0]c;
6
7     assign c = (~s) ? a : b ;
8
9 endmodule
```

PC ADDER

```
1 module PC_Adder (a,b,c);
2
3     input [31:0]a,b;
4     output [31:0]c;
5
6     assign c = a + b;
7
8 endmodule
```

PC(PROGRAM COUNTER)

```
1 module PC_Module(clk,rst,PC,PC_Next);
2     input clk,rst;
3     input [31:0]PC_Next;
4     output [31:0]PC;
5     reg [31:0]PC;
6
7     always @(posedge clk)
8     begin
9         if(~rst)
10            PC <= {32{1'b0}};
11        else
12            PC <= PC_Next;
13    end
14 endmodule
```

INSTRUCTION MEMORY

```
1 module Instruction_Memory(rst,A,RD);
2
3     input rst;
4     input [31:0]A;
5     output [31:0]RD;
6
7     reg [31:0] mem [1023:0];
8
9     assign RD = (~rst) ? {32{1'b0}} : mem[A[31:2]];
10
11     initial begin
12         //$readmemh("memfile.hex",mem);|
13         mem[0] = 32'hFFC4A303;
14         //mem[1] = 32'h0064A423;
15         //mem[2] = 32'h0062E223;
16     end
17 endmodule
```

REGISTER FILE

```

1 module Register_File(clk,rst,WE3,WD3,A1,A2,A3,RD1,RD2);
2
3     input clk,rst,WE3;
4     input [4:0]A1,A2,A3;
5     input [31:0]WD3;
6     output [31:0]RD1,RD2;
7
8     reg [31:0] Register [31:0];
9
10    always @ (posedge clk)
11    begin
12        if(WE3)
13            Register[A3] <= WD3;
14    end
15
16    assign RD1 = (~rst) ? 32'd0 : Register[A1];
17    assign RD2 = (~rst) ? 32'd0 : Register[A2];
18
19    initial begin
20        // Register[5] = 32'h00000005;
21        // Register[6] = 32'h00000004;
22        Register[9] = 32'h00000020;
23    end
24
25 endmodule
26

```

SIGN EXTEND

```

1 module Sign_Extend (In,Imm_Ext,ImmSrc);
2
3     input [31:0]In;
4     input ImmSrc;
5     output [31:0]Imm_Ext;
6
7     assign Imm_Ext = (ImmSrc == 1'b1) ? ({20{In[31]}},In[31:25],In[11:7]):
8                                         {{20{In[31]}},In[31:20]};
9
10 endmodule

```

DATA MEMORY

```

1 module Data_Memory(clk,rst,WE,WD,A,RD);
2
3     input clk,rst,WE;
4     input [31:0]A,WD;
5     output [31:0]RD;
6
7     reg [31:0] mem [1023:0];
8
9     always @ (posedge clk)
10    begin
11        if(WE)
12            mem[A] <= WD;
13    end
14
15    assign RD = (~rst) ? 32'd0 : mem[A];
16
17    initial begin
18        mem[28] = 32'h00000020;
19    end
20
21 |
22 endmodule

```

SINGLE CYCLE TOP

```
1 // Code your design here
2 `include "PC.v"
3 `include "Instruction_Memory.v"
4 `include "Register_File.v"
5 `include "Sign_Extend.v"
6 `include "ALU.v"
7 `include "Control_Unit_Top.v"
8 `include "Data_Memory.v"
9 `include "PC_Adder.v"
10 `include "Mux.v"
11
12 module Single_Cycle_Top(clk,rst);
13
14     input clk,rst;
15
16     wire [31:0] PC_Top,RD_Instr,RD1_Top,Imm_Ext_Top,ALUResult,ReadData,PCPlus4,RD2_Top,SrcB,Result;
17     wire RegWrite,MemWrite,ALUSrc,ResultSrc;
18     wire [1:0] ImmSrc;
19     wire [2:0] ALUControl_Top;
20
21     PC_Module PC(
22         .clk(clk),
23         .rst(rst),
24         .PC(PC_Top),
25         .PC_Next(PCPlus4)
26     );
27
28     PC_Adder PC_Adder(
29         .a(PC_Top),
30         .b(32'd4),
31         .c(PCPlus4)
32     );
33
34     Instruction_Memory Instruction_Memory(
35         .rst(rst),
36         .A(PC_Top),
37         .RD(RD_Instr)
38     );
39
40     Register_File Register_File(
41         .clk(clk),
42         .rst(rst),
43         .WE3(RegWrite),
44         .WD3(Result),
45         .A1(RD_Instr[19:15]),
```

```
46         .A2(RD_Instr[24:20]),
47         .A3(RD_Instr[11:7]),
48         .RD1(RD1_Top),
49         .RD2(RD2_Top)
50     );
51
52     Sign_Extend Sign_Extend(
53         .In(RD_Instr),
54         .ImmSrc(ImmSrc[0]),
55         .Imm_Ext(Imm_Ext_Top)
56     );
57
58     Mux Mux_Register_to_ALU(
59         .a(RD2_Top),
60         .b(Imm_Ext_Top),
61         .s(ALUSrc),
62         .c(SrcB)
63     );
64
65     ALU ALU(
66         .A(RD1_Top),
67         .B(SrcB),
68         .Result(ALUResult),
69         .ALUControl(ALUControl_Top),
70         .Overflow(),
71         .Carry(),
72         .Zero(),
73         .Negative()
74     );
75
76     Control_Unit_Top Control_Unit_Top(
77         .Op(RD_Instr[6:0]),
78         .RegWrite(RegWrite),
79         .ImmSrc(ImmSrc),
80         .ALUSrc(ALUSrc),
81         .MemWrite(MemWrite),
82         .ResultSrc(ResultSrc),
83         .PCSrc(),
84         .funct3(RD_Instr[14:12]),
85         .funct7(RD_Instr[6:0]),
86         .ALUControl(ALUControl_Top)
87     );
88
89     Data_Memory Data_Memory(
90         .clk(clk),
```

```

91         .rst(rst),
92         .WE(MemWrite),
93         .WD(RD2_Top),
94         .A(ALUResult),
95         .RD(ReadData)
96     );
97
98     Mux Mux_DataMemory_to_Register(
99         .a(ALUResult),
100        .b(ReadData),
101        .s(ResultSrc),
102        .c(Result)
103    );
104
105 endmodule

```

TESTBENCH

```

1 module Single_Cycle_Top_Tb ();
2
3     reg clk=1'b1,rst;
4
5     Single_Cycle_Top Single_Cycle_Top(
6         .clk(clk),
7         .rst(rst)
8     );
9
10    initial begin
11        $dumpfile("Single Cycle.vcd");
12        $dumpvars(0);
13    end
14
15    always
16    begin
17        clk = ~ clk;
18        #50;
19    end
20
21    initial
22    begin
23        rst <= 1'b0;
24        #150;
25
26        rst <= 1'b1;
27        #450;
28        $finish;
29    end
30 endmodule

```

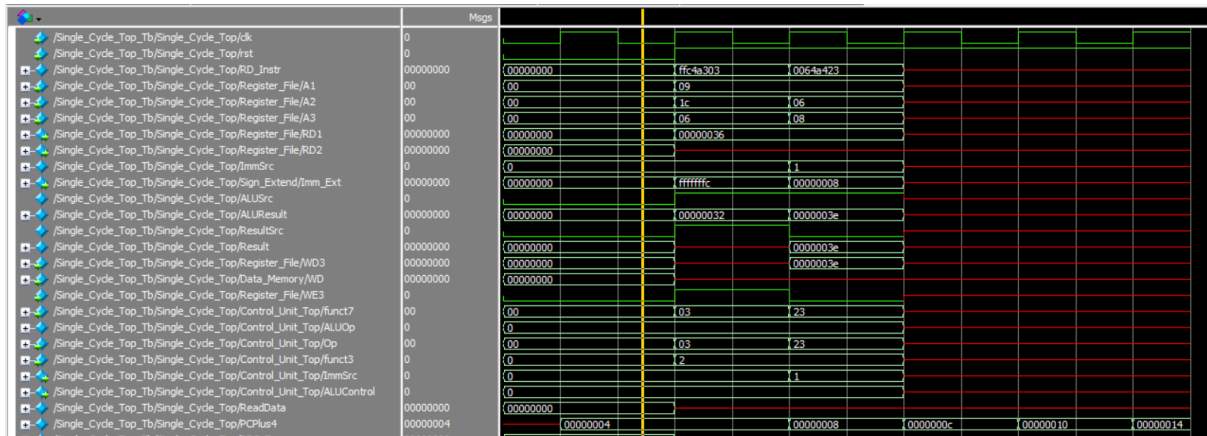


FIG: SIMULATION RESULT FOR LOAD WORD AND STORE WORD

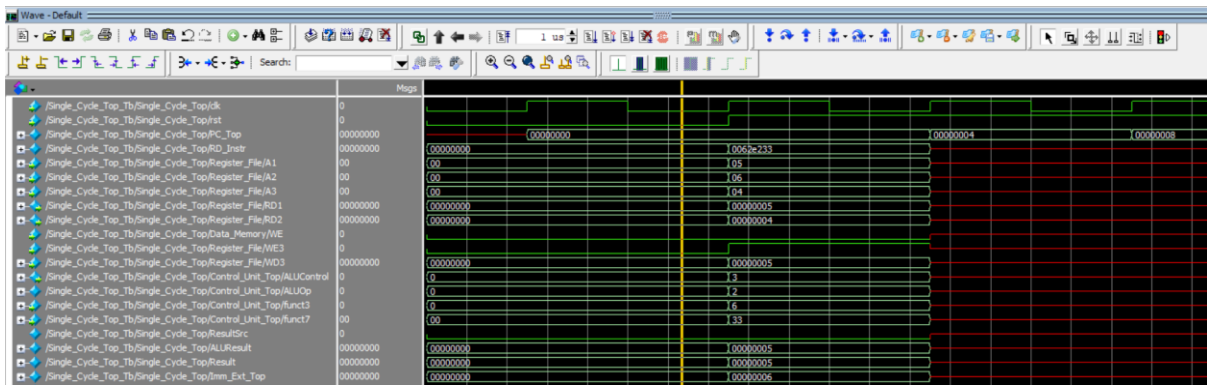


FIG: SIMULATION RESULT FOR R-TYPE INSTRUCTION

REFERENCES

- Computer Architecture: A Quantitative Approach-Book by David A Patterson and John L. Hennessy
- Digital Design and Computer Architecture, RISC-V Edition
Book by David Harris (Author), Sarah L. Harris (Author)
- [Lecture videos](#) by Sarah and David Harris